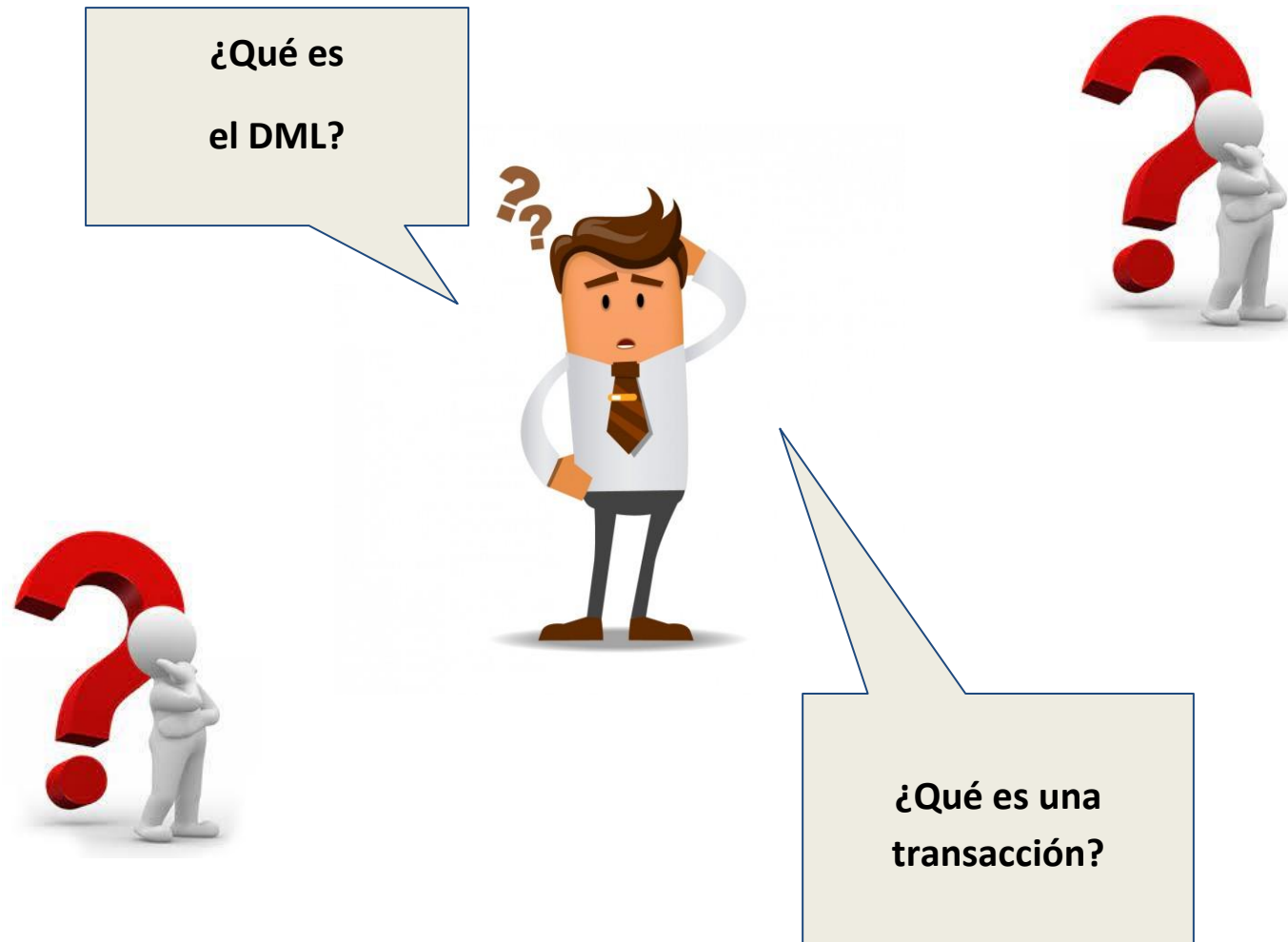


Tema 6. Lenguaje de manipulación de datos

1. Introducción
2. Inserción de datos
3. Actualización de datos
4. Borrado de registros
5. Transacciones



1. Introducción



1. Introducción

¿Qué es
el DML?

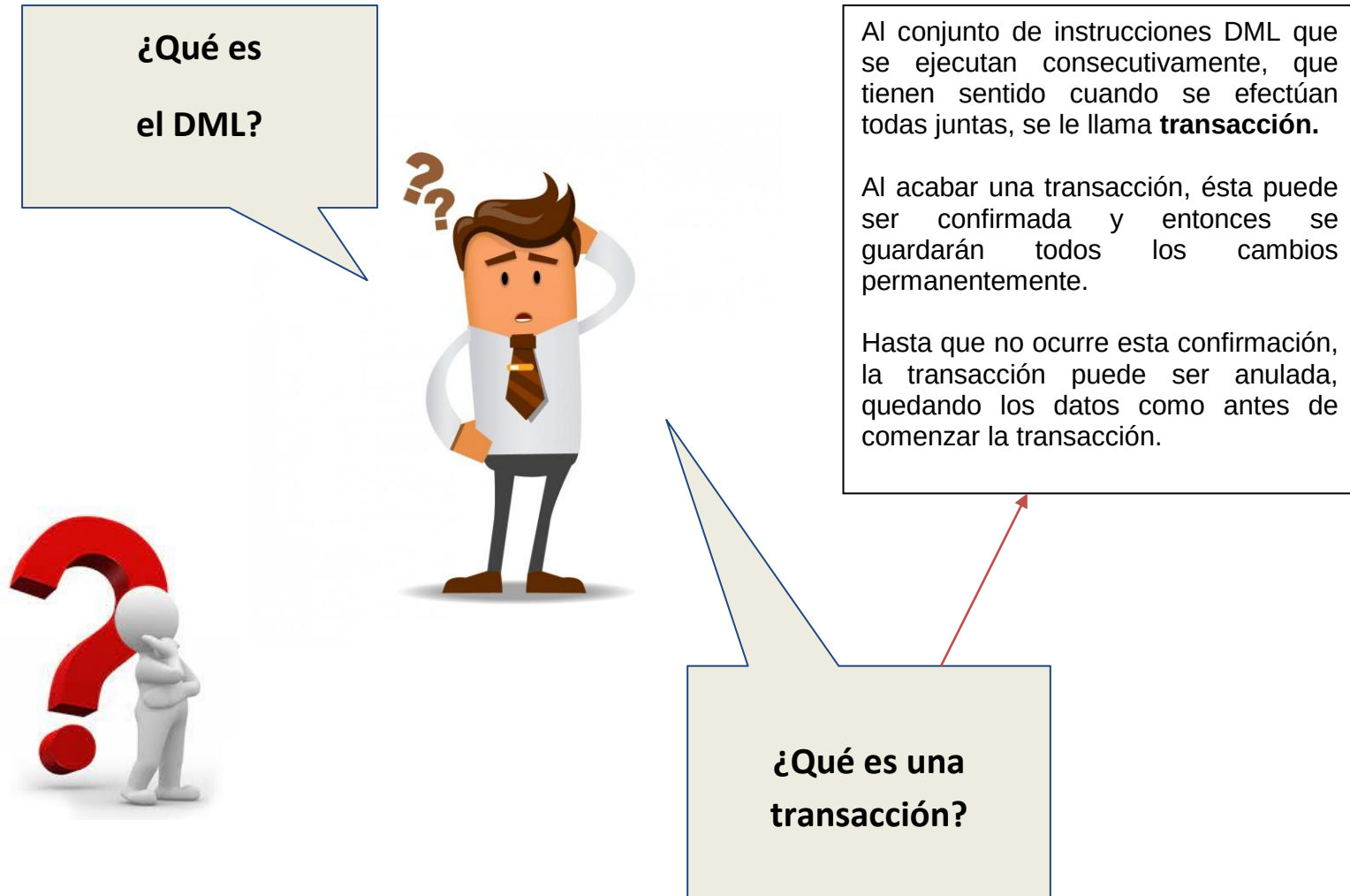
El Lenguaje de manipulación de datos o DML debido a sus siglas en inglés, es una de las partes fundamentales del lenguaje SQL.

El **DML (Data Manipulation Language)** lo forman las instrucciones capaces de modificar los datos de las tablas: insertar nuevas filas (*INSERT*), modificar los valores existentes (*UPDATE*) y eliminar filas (*DELETE*).



¿Qué es una
transacción?

1. Introducción



2. Inserción de datos

La adición de datos a una tabla se realiza mediante la instrucción INSERT. Su sintaxis básica es:

Sintaxis 1

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [PARTITION (partition_list)] [(col,...)]
{VALUES | VALUE} ({expr | DEFAULT},...),(...),...
[ ON DUPLICATE KEY UPDATE
  col=expr
  [, col=expr] ... ] [RETURNING select_expr
  [, select_expr ...]]
```

Sintaxis 2

```
INSERT [LOW_PRIORITY | DELAYED | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [PARTITION (partition_list)]
SET col={expr | DEFAULT}, ...
[ ON DUPLICATE KEY UPDATE
  col=expr
  [, col=expr] ... ] [RETURNING select_expr
  [, select_expr ...]]
```

Estas opciones **permiten la inserción de nuevas filas basadas en valores especificados explícitamente**.

En la sintaxis 1, la lista de valores que siguen a VALUES (o VALUE) tiene que estar entre paréntesis. Los valores se tienen que listar en el mismo orden que la lista de columnas. La lista de columnas es opcional. Especifica los valores que serán explícitamente insertados y en qué orden. Si esta cláusula no se especifica, todos los valores deben ser explícitamente especificados en el mismo orden en el que son listados en la tabla base (es el orden de columnas según las devuelve el comando DESCRIBE tabla).

Hay que tener cuidado para que coincida el tipo de dato de cada columna con el valor asignado. Es bastante arriesgado trabajar así, ya que, si la tabla evoluciona en el tiempo con nuevas columnas, este orden no funcionará (el sistema indicará que faltan valores). **Es preferible indicar siempre las columnas que se van a rellenar.**

Es posible especificar en una única sentencia más de una lista de valores a insertar, lo que redundaría en una mejor optimización.

2. Inserción de datos

Para inserciones de una única fila, la sintaxis 2 puede ser más simple ya que no se necesita recordar el orden de las columnas. En este caso, todos los valores se especifican como *col = expr*.

Ejemplos:

- Especificando los nombres de las columnas:
`INSERT INTO person (first_name, last_name) VALUES ('John', 'Doe');`
- Insertando más de una fila de una vez:
`INSERT INTO tbl_name VALUES (1, "row 1"), (2, "row 2");`
- Utilizando la cláusula SET:
`INSERT INTO person SET first_name = 'John', last_name = 'Doe';`

Los campos no rellenados explícitamente con la orden **INSERT**, se rellenan con su valor por defecto (**DEFAULT**) o bien con **NULL** si no se indicó valor alguno. Si algún campo tiene restricción de obligatoriedad (**NOT NULL**), **ocurrirá un error** si no rellenamos el campo con algún valor.

Ejemplo: supongamos que tenemos una tabla de clientes cuyos campos son: *dni, nombre, apellido1, apellido2, localidad y dirección*; y supongamos también que ése es el orden de creación de los campos de esa tabla, que la localidad tiene como valor por defecto *Palencia* y la dirección no tiene valor por defecto. En ese caso estas dos instrucciones son equivalentes:

```
INSERT INTO clientes VALUES ('11111111', 'Pedro', 'Gutiérrez', 'Crespo', DEFAULT, NULL);
```

```
INSERT INTO clientes (dni, nombre, apellido1, apellido2) VALUES ('11111111', 'Pedro', 'Gutiérrez', 'Crespo');
```

Son equivalentes puesto que en la segunda instrucción los campos no indicados se rellenan con su valor por defecto (Palencia y NULL, pues la dirección no tiene valor por defecto como tal). La palabra **DEFAULT** fuerza a utilizar ese valor por defecto.

***HACER EJERCICIOS DEL 1 AL 4**

2. Inserción de datos

INSERCIÓN DE DATOS TOMÁNDOLOS DE LOS YA EXISTENTES EN LA BASE DE DATOS

Se pueden añadir en una misma sentencia *INSERT* las filas devueltas por una consulta *SELECT*. Las columnas de la salida de la consulta deben coincidir en número y tipo con las columnas que queremos rellenar.

```
INSERT [LOW_PRIORITY | HIGH_PRIORITY] [IGNORE]
[INTO] tbl_name [PARTITION (partition_list)] [(col,...)]
SELECT ...
[ ON DUPLICATE KEY UPDATE
  col=expr
  [, col=expr] ... ] [RETURNING select_expr
[, select_expr ...]]
```

Ejemplos:

Introducir en la tabla ANTIGUOS al alumno Juan que está en la tabla ALUM, pero considerando que se ha cambiado de Coslada a Madrid:

```
INSERT INTO antiguos
SELECT nombre, edad, 'MADRID' FROM alum WHERE nombre='JUAN';
```

***HACER EJERCICIOS DEL 5 AL 10**

2. Inserción de datos

MODIFICADORES DE LA SENTENCIA INSERT

- ✚ **INSERT DELAYED.** Se especifica la palabra clave *DELAYED*, el servidor coloca la fila o filas a insertar en un búfer, y el cliente lanza la sentencia *INSERT DELAYED* y puede continuar. Si la tabla está ocupada, el servidor guarda las filas. Cuando la tabla queda libre, empieza a insertar filas, verificando periódicamente para ver si hay nuevas peticiones de lectura para la tabla. Si las hay, la cola de inserción retardada se suspende hasta que la tabla quede libre de nuevo.
- ✚ **HIGH_PRIORITY and LOW_PRIORITY.** Si se especifica la palabra clave *LOW_PRIORITY*, la ejecución de la sentencia *INSERT* se retrasa hasta que no haya clientes leyendo de la tabla. Si se especifica la palabra clave *HIGH_PRIORITY*, se anula el efecto de la opción *--low-priority-updates* si el servidor fue arrancado con esa opción.
- ✚ **INSERT IGNORE.** Si se especifica la palabra *IGNORE* en una sentencia *INSERT*, los errores que se produzcan mientras se ejecuta la sentencia se tratan con avisos y no como errores en sí. Por ejemplo, sin *IGNORE* una fila que duplique un valor de clave *PRIMARY* o *UNIQUE* existente en la tabla provocará un error y la sentencia será abortada. Con *IGNORE*, el error se ignora y la fila no será insertada. Las conversiones de datos que produzcan errores abortarán la sentencia si no se especifica *IGNORE*. Con *IGNORE*, los valores inválidos se ajustarán al valor más próximos y se insertarán; se producirán avisos, pero la sentencia no se aborta.
- ✚ **INSERT ON DUPLICATE KEY UPDATE.** Si se especifica la cláusula *ON DUPLICATE KEY UPDATE* y se inserta una fila que puede provocar un valor duplicado en una clave *PRIMARY* o *UNIQUE*, se realiza un *UPDATE* (actualización) de la fila antigua, es decir, permite transformar en *UPDATE* la inserción de una fila que generaría un duplicado en una clave primaria o única. Si se especifica esta opción, los modificadores *IGNORE* y *DELAYED*, si existen, son ignorados.

2. Inserción de datos

SENTENCIA REPLACE

REPLACE es una extensión de MariaDB del SQL estándar y funciona exactamente igual que *INSERT*, excepto que, si una fila antigua de la tabla tiene el mismo valor que una fila nueva para una clave primaria o un índice único, la fila anterior se elimina antes de insertar la nueva fila.

Si la tabla tiene más de una clave única, es posible que la nueva fila entre en conflicto con más de una fila. En este caso, se eliminarán todas las filas en conflicto.

Su sintaxis es:

```
REPLACE [LOW_PRIORITY | DELAYED]
[INTO] tbl_name [PARTITION (partition_list)] [(col,...)]
{VALUES | VALUE} ({expr | DEFAULT},...),(...),...
[RETURNING select_expr
[, select_expr ...]]
```

Más información: <https://mariadb.com/kb/en/replace/>

3. Actualización de datos

La modificación de los datos de los registros lo implementa la instrucción **UPDATE**. Sintaxis:

```
UPDATE [LOW_PRIORITY] [IGNORE] table_reference
  [PARTITION (partition_list)]
  SET col1={expr1|DEFAULT} [,col2={expr2|DEFAULT}] ...
  [WHERE where_condition]
  [ORDER BY ...]
  [LIMIT row_count]
```

Se modifican las columnas indicadas en el apartado **SET** con los valores indicados. Cada valor puede tomar el valor dado en la expresión o, si se utiliza la palabra reservada **DEFAULT**, tomará el valor por defecto de la columna indicada.

La cláusula **WHERE** permite especificar qué registros serán modificados. **Si no se especifica cláusula WHERE, se actualizarán todas las filas.**

Si se especifica la cláusula **ORDER BY**, las filas son actualizadas en el orden en que son especificadas. La cláusula **LIMIT** fija un límite al número de filas que pueden ser actualizadas, por lo que el **ORDER BY**, en este caso, sólo es interesante cuando se utiliza con la cláusula **LIMIT**.

Es aconsejable, antes del propio **UPDATE**, hacer un **SELECT** previo para visualizar las filas que serán afectadas por la actualización de datos.

Ejemplo:

Actualiza la provincia de los clientes de Orense para que aparezca como Ourense.

```
UPDATE clientes SET provincia='Ourense'
  WHERE provincia='Orense';
```

3. Actualización de datos

Ejemplo:

Incrementa los precios en un 16%.

```
UPDATE productos SET precio=precio*1.16;
```

La expresión para el valor puede ser todo lo compleja que se desee, como, por ejemplo, utilizar funciones de fecha para conseguir que los partidos que se jugaban hoy, pasen a jugarse dentro de 7 días:

```
UPDATE partidos SET fecha=ADDDATE(SYSDATE(),7)
WHERE fecha=SYSDATE();
```

MariaDB también admite una sintaxis de actualización en varias tablas combinadas.

Ejemplo:

Actualizar en una única sentencia *UPDATE* las tablas colección y libro.

```
UPDATE libro lib JOIN colección col ON (lib.id_coleccion = col.id) SET
-- aumentar el precio de la colección
col.precio_siniva = col.precio_siniva + 0.47,
-- poner los libros (de la colección) en promoción
lib.id_promocion = 3
WHERE col.nombre NOT LIKE '%infor%' and lib.id_promocion IS NULL;
```

***HACER LOS EJERCICIOS 11 Y 12**

3. Actualización de datos

ACTUALIZACIÓN TOMADO LOS DATOS DE LOS YA EXISTENTES EN LA BASE DE DATOS

Para esta situación se utiliza la sintaxis:

```
UPDATE tabla
SET col1 = (SELECT col1 FROM tabladatos1) [,col2 = ((SELECT col2 FROM tabladatos2))] ...
WHERE condición;
```

En este caso, **el *SELECT* debe devolver un único valor, y con él se actualizará la columna correspondiente.**

También **la cláusula *WHERE* puede contener una subconsulta.** La sintaxis es:

```
UPDATE tabla
SET campo1=expresion1, campo2=expresion2, ...
WHERE campo3 = (SELECT columna ...)
```

Ejemplo:

Los empleados del departamento de contabilidad cambian su sueldo al doble del salario del empleado Fernández y su apellido a minúsculas.

```
UPDATE emple SET apellido = LOWER(apellido),
salario = (SELECT salario*2 FROM emple WHERE UPPER(apellido) = 'FERNANDEZ') WHERE dept_no = (SELECT dept_no FROM
depart WHERE UPPER(dnombre) = 'CONTABILIDAD');
```

***HACER LOS EJERCICIOS DEL 13 AL 16**

3. Actualización de datos

MODIFICADORES DE LA SENTENCIA UPDATE

- 🔗 **LOW_PRIORITY.** Cuando se utiliza este modificador, la ejecución del *UPDATE* se retrasa hasta que ningún otro cliente esté leyendo de la tabla. Esta orden sólo afecta a **motores de almacenamiento** que utilizan solo bloqueo a nivel de tabla (como MyISAM, pero no con InnoDB, que es el utilizado por defecto).
- 🔗 **IGNORE.** Permite ignorar los errores vinculados a la modificación de filas, que provocarían un duplicado en una clave primaria o única. Si eso se produce, no se modifica la fila, no se muestra ningún mensaje de error y la modificación continúa en las otras filas.

Los motores de almacenamiento determinan la manera en que los datos se almacenan y qué operaciones son posibles sobre los datos. Los más utilizados son:

- **InnoDB** – Permite gestionar claves foráneas, así como transacciones y bloqueos a nivel de fila. Es el motor predeterminado.
- **MyISAM** – Motor muy eficiente, pero no gestiona transacciones. Se usa con frecuencia en tablas que se usan principalmente en modo lectura.
- **ARCHIVE** – Motor que almacena los datos en un formato comprimido. Es interesante para tablas voluminosas, pero la compresión/descompresión ralentiza las operaciones de lectura o actualización.
- **MEMORY** – Motor que almacena los datos en memoria. Es interesante para las tablas temporales, pero los datos se pierden cuando el servidor se detiene.

4. Borrado de registros

Se realiza mediante la instrucción **DELETE**. Su sintaxis es:

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE]
FROM tbl_name [PARTITION (partition_list)]
[WHERE where_condition]
[ORDER BY ...]
[LIMIT row_count]
[RETURNING select_expr
[, select_expr ...]]
```

Es más sencilla que las anteriores, elimina los registros de la tabla que cumplan la condición indicada. **Es importante tener en cuenta la cláusula *WHERE*, porque si se olvida pueden borrarse todas las filas de una tabla.** La condición puede incluir una subconsulta.

Para evitar borrados no deseados, un consejo: probar previamente con un **SELECT** la cláusula *WHERE* para verificar el número de filas afectadas.

Si se especifica la cláusula **ORDER BY**, las filas son borradas en el orden en que son especificadas. La cláusula **LIMIT** fija un límite al número de filas que pueden ser borradas, por lo que el *ORDER BY*, en este caso, sólo es interesante cuando se utiliza con la cláusula *LIMIT*.

MariaDB también admite una sintaxis de borrado en varias tablas combinadas. En este caso, *ORDER BY* y *LIMIT* no pueden utilizarse.

Ejemplo:

Borrar los empleados de la sección 23

```
DELETE FROM empleados
WHERE seccion=23;
```

4. Borrado de registros

Ejemplos:

Borrar las cuentas de las sucursales de Toledo.

```
DELETE FROM cuentas
WHERE cod_sucur IN (SELECT cod_sucur FROM sucursales WHERE loc_suc='Toledo');
```

Borrar la fila de la colección con el identificador más alto.

```
DELETE FROM coleccion ORDER BY id DESC LIMIT 1;
```

Borrar las cuentas de la sucursal de menor código de banco:

```
DELETE FROM cuentas WHERE cod_sucur IN
(SELECT cod_sucur FROM sucursales WHERE cod_banco=(SELECT MIN(cod_banco) FROM sucursales));
```

MariaDB también admite una sintaxis de borrado en varias tablas combinadas. La cláusula *FROM* especifica las tablas en las que deben eliminarse las filas. La cláusula *USING* especifica todas las tablas utilizadas en la combinación (puede haber tablas presentes en este cláusula y no en la *FROM* para indicar tablas utilizadas en la combinación pero en las que no hay filas que suprimir). Si se utiliza un alias en la cláusula *USING*, ese alias debe utilizarse en la cláusula *FROM*.

Ejemplo:

Borrar la fila del libro con isbn '2-7460-3104-3', así como las filas unidad en las tablas tema_libro y autor_libro (ya que esas filas carecerían ya de sentido).

```
DELETE FROM lib, tel, aul – alias definidos en la cláusula USING
USING libro lib, tema_libro tem, autor_libro aul
WHERE lib.id = tel.id_libro AND lib.id = aul.id_libro AND lib.isbn='2-7460-3104-3';
```

***HACER LOS EJERCICIOS DEL 17 AL 22**

4. Borrado de registros

MODIFICADORES DE LA SENTENCIA DELETE

- 🔗 **LOW_PRIORITY.** Cuando se utiliza este modificador, la ejecución del *DELETE* se retrasa que todos los *SELECT* sean hechos antes de comenzar la sentencia. Esta orden sólo afecta a motores de almacenamiento que utilizan solo bloqueo a nivel de tabla (como MyISAM, pero no con InnoDB).
- 🔗 **QUICK.** Indica al motor de almacenamiento que debe esperar a que se eliminen muchas filas. El motor de almacenamiento puede hacer cosas para acelerar el *DELETE*, como ignorar la fusión de bloques de datos hasta que se eliminen todas las filas del bloque (en lugar de cuando un bloque está medio lleno). Esta orden no afecta a todos los motores de almacenamiento (sí funciona en MyISAM, pero no en InnoDB).
- 🔗 **IGNORE.** Permite ignorar los errores vinculados a la eliminación de filas que provocaría un error (violación de clave externa, por ejemplo). Si esto se produce, no se elimina la fila, no se muestra ningún mensaje de error y la eliminación continúa en las otras filas.

SENTENCIA TRUNCATE

Esta sentencia vacía una tabla de datos completamente. Su sintaxis es:

```
TRUNCATE [TABLE] tbl_name  
[WAIT n | NOWAIT]
```

Lógicamente, esta orden es **equivalente a una sentencia *DELETE* que borre todas las filas**, pero hay **ligeras diferencias** bajo ciertas circunstancias, como con el uso del motor de almacenamiento InnoDB.

Más información: <https://mariadb.com/kb/en/truncate-table/>

5. Transacciones

Como se ha comentado anteriormente, **una transacción está formada por una serie de instrucciones DML**. Se define **cuando un conjunto de sentencias debe ejecutarse por completo o no debe ejecutarse nada**.

El típico ejemplo es la transferencia de dinero de una cuenta a otra. El importe debe restarse del saldo de una cuenta (mediante *UPDATE*) y añadirse al saldo de otra cuenta (también mediante *UPDATE*). Si queremos que la bd refleje la realidad, es decir, si queremos que contenga información consistente, las dos operaciones anteriores deben hacerse las dos en conjunto o ninguna, por lo tanto, sería bueno definir las como transacción.

Una transacción SQL **finaliza con un `COMMIT`**, para **aceptar todos los cambios que la transacción ha realizado** en la base de datos, o un **`ROLLBACK`** para **deshacerlos**.

En MySQL Workbench se tienen los iconos para anular (ROLLBACK) o confirmar (COMMIT) una transacción.



Para habilitar estos botones se tiene el botón **“Toggle autocommit mode”**.

MariaDB permite realizar transacciones en sus tablas si hacemos uso del motor de almacenamiento *InnoDB*, pero no si utilizásemos *MyISAM*, por ejemplo.

El uso de transacciones permite realizar operaciones de forma segura y recuperar datos si se produce algún fallo en el servidor durante la transacción, pero, por otro lado, las transacciones pueden aumentar el tiempo de ejecución de las instrucciones.

5. Transacciones

Las transacciones deben cumplir las 4 propiedades **ACID**:

- **Atomicidad (Atomicity)**. Indica que una transacción es indivisible, o **se ejecutan todas las sentencias o no se ejecuta ninguna**.
- **Consistencia (Consistency)**. Asegura que después de una transacción la base de datos estará en un **estado válido y consistente**.
- **Aislamiento (Isolation)**. Garantiza que cada transacción está aislada del resto de transacciones y que **el acceso a los datos se hará de forma exclusiva**. Por ejemplo, si una transacción quiere acceder de forma concurrente a los datos que están siendo utilizados por otra transacción, no podrá hacerlo hasta que la primera haya terminado.
- **Durabilidad (Durability)**. Indica que los **cambios** que realiza una transacción sobre la base de datos son **permanentes**.

Algunos SGBD, como MariaDB si se trabaja con el motor *InnoDB* tienen activado por defecto la variable **AUTOCOMMIT**, que indica que **automáticamente se aceptan todos los cambios realizados** después de la ejecución de una sentencia SQL y **no es posible deshacerlos**.

Si se desea, se puede configurar dicha variable para hacer que el sistema trabaje con transacciones de manera explícitas.

Se puede consultar el valor de *AUTOCOMMIT* ejecutando *SELECT @@AUTOCOMMIT;*

Para desactivar la variable, hacemos ***SET AUTOCOMMIT = 0;***

Si hacemos esto, siempre tendríamos una transacción abierta y los cambios sólo se aplicarían en la base de datos ejecutando la sentencia COMMIT de forma explícita. Desde el momento en que la validación automática está desactivada, no es necesario iniciar explícitamente las transacciones; una nueva transacción se inicia implícitamente al final de cada transacción.

Para activar nuevamente la variable, haríamos *SET AUTOCOMMIT = 1;*

5. Transacciones

START TRANSACTION, COMMIT y ROLLBACK

Los pasos para realizar una transacción en MariaDB son:

- Indicar que se va a realizar una transacción con la sentencia **START TRANSACTION, BEGIN ó BEGIN WORK**
 - Es **aconsejable utilizar START TRANSACTION** ya que es la que se ajusta al estándar SQL.
 - Cuando una transacción es iniciada explícitamente de esta manera, la validación automática es desactivada el tiempo de la transacción; al final de la transacción, la validación automática vuelve a su estado anterior.
- Realizar las **operaciones de manipulación de datos sobre la base de datos** (insertar, actualizar o borrar filas).
- Si las operaciones se han realizado correctamente y queremos que los cambios se apliquen de forma permanente sobre la base de datos, **usaremos la sentencia COMMIT**. Sin embargo, si durante las operaciones ocurre algún error y no queremos aplicar los cambios realizados, **podemos deshacerlos con la sentencia ROLLBACK**.

Las sentencias del lenguaje de definición de datos (DDL), como **CREATE, ALTER, DROP, no pueden ser anuladas**. Por otro lado, estas sentencias validan implícitamente la transacción en curso. Igualmente, activar la validación automática (**SET AUTOCOMMIT = 1**) o iniciar una transacción valida implícitamente también la transacción en curso. La sintaxis utilizada es:

```
START TRANSACTION [transaction_property [, transaction_property] ...] | BEGIN [WORK]
COMMIT [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
ROLLBACK [WORK] [AND [NO] CHAIN] [[NO] RELEASE]
SET autocommit = {0 | 1}

transaction_property:
    WITH CONSISTENT SNAPSHOT
  | READ WRITE
  | READ ONLY
```

La sentencia **SET autocommit** deshabilita o habilita el modo de autocommit por defecto para la sesión actual. La sentencia **AND CHAIN** provoca que una **nueva transacción comience** tan pronto como la actual finalice. La sentencia **RELEASE** hace que el **servidor desconecte la sesión** del cliente actual después de terminar la transacción actual.

5. Transacciones

START TRANSACTION, COMMIT y ROLLBACK

El modo de acceso especifica si la transacción tendrá permisos para escribir datos o no. **Por defecto**, las transacciones tienen el **modo READ WRITE**. El modo **READ ONLY** permite al motor de almacenamiento aplicar optimizaciones que no pueden ser utilizados por las transacciones que escriban datos (con la excepción de sentencias DDL (CREATE, ALTER, DROP) en tablas temporales).

Ejemplos:

1)

```
START TRANSACTION;
SELECT @A:=SUM(salary) FROM table1 WHERE type=1;
UPDATE table2 SET summary=@A WHERE type=1;
COMMIT;
```

2)

```
DROP DATABASE IF EXISTS test;
CREATE DATABASE test CHARACTER SET utf8mb4;
USE test;
CREATE TABLE cliente (
  id INT UNSIGNED PRIMARY KEY,
  nombre CHAR (20));
```

```
START TRANSACTION;
INSERT INTO cliente VALUES (1, 'Pepe');
COMMIT;
```

-- 1. ¿Qué devolverá esta consulta?

```
SELECT *
FROM cliente;
-- Inserta la fila "1 - pepe"
```

```
SET AUTOCOMMIT=0;
INSERT INTO cliente VALUES (2, 'Maria');
INSERT INTO cliente VALUES (20, 'Juan');
DELETE FROM cliente WHERE nombre = 'Pepe';
```

-- 2. ¿Qué devolverá esta consulta? -->

```
SELECT *
FROM cliente;
-- Inserta las filas "2 - María" y "20 - Juan" y borra la fila de Pepe.
```

ROLLBACK;

-- 3. ¿Qué devolverá esta consulta?

```
SELECT *
FROM cliente;
-- Deshace los cambios producidos anteriormente (las 2
inserciones de María y Juan y el borrado de Pepe). Por lo tanto,
vuelve a mostrarse la fila "1 - Pepe" en la tabla.
```

3)

```
DROP DATABASE IF EXISTS test;
CREATE DATABASE test CHARACTER SET utf8mb4;
USE test;
CREATE TABLE cuentas (
  id INTEGER UNSIGNED PRIMARY KEY,
  saldo DECIMAL(11,2) CHECK (saldo >= 0)
);
INSERT INTO cuentas VALUES (1, 1000);
INSERT INTO cuentas VALUES (2, 2000);
INSERT INTO cuentas VALUES (3, 0);
```

-- 1. Consultamos el estado actual de las cuentas

```
SELECT *
FROM cuentas;
-- Devuelve 3 filas: "1 - 1000.00", "2 - 2000.00", "3 - 0.00"
```

-- 2. Suponga que queremos realizar una transferencia de dinero entre dos cuentas bancarias con la siguiente transacción:

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;
```

-- 3. ¿Qué devolverá esta consulta?

```
SELECT *
FROM cuentas; -- "1 - 900.00", "2 - 2100.00", "3 - 0.00"
```

-- 4. Suponga que queremos realizar una transferencia de dinero entre dos cuentas bancarias con la siguiente transacción y una de las dos cuentas no existe:

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 9999;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;
```

-- 5. ¿Qué devolverá esta consulta?

```
SELECT * FROM cuentas;
-- "1 - 900.00", "2 - 2200.00", "3 - 0.00"
```

-- 6. Suponga que queremos realizar una transferencia de dinero entre dos cuentas bancarias con la siguiente transacción y la cuenta origen no tiene saldo:

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 3;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
COMMIT;
```

-- Se produce un error: "CONSTRAINT cuentas.saldo falló" (el CHECK de saldo>=0) no se cumple.

-- 7. ¿Qué devolverá esta consulta?

```
SELECT *
FROM cuentas;
-- "1 - 900.00", "2 - 2200.00", "3 - 0.00"
```

5. Transacciones

SAVEPOINT, ROLLBACK TO SAVEPOINT y RELEASE SAVEPOINT

Si se trabaja con el motor de almacenamiento InnoDB, es posible hacer uso de las sentencias:

- **SAVEPOINT:** permite establecer un punto de recuperación dentro de la transacción, utilizando un identificador. Si en una transacción existen dos **SAVEPOINT** con el mismo nombre, sólo se tendrá en cuenta el último que se ha definido.
- **ROLLBACK TO SAVEPOINT:** permite hacer un **ROLLBACK** deshaciendo sólo las instrucciones que se han ejecutado hasta el **SAVEPOINT** que se indique.
- **RELEASE SAVEPOINT:** elimina un **SAVEPOINT**.

La sintaxis de estas sentencias es:

```
SAVEPOINT identifier
ROLLBACK [WORK] TO [SAVEPOINT] identifier
RELEASE SAVEPOINT identifier
```

Todos los puntos de retorno de una transacción son eliminados cuando la transacción se termina (**COMMIT** o **ROLLBACK**).

Ejemplo:

```
DROP DATABASE IF EXISTS test;
CREATE DATABASE test CHARACTER SET utf8mb4;
USE test;
CREATE TABLE producto (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  precio DOUBLE
);

INSERT INTO producto (id, nombre) VALUES (1, 'Primero');
INSERT INTO producto (id, nombre) VALUES (2, 'Segundo');
INSERT INTO producto (id, nombre) VALUES (3, 'Tercero');
```

```
-- 1. Comprobamos las filas que existen en la tabla
SELECT *
FROM producto;
-- "1 - Primero", "2 - Segundo", "3 - Tercero"

-- 2. Ejecutamos una transacción que incluye un SAVEPOINT
START TRANSACTION;
INSERT INTO producto (id, nombre) VALUES (4, 'Cuarto');
SAVEPOINT sp1;
INSERT INTO producto (id, nombre) VALUES (5, 'Cinco');
INSERT INTO producto (id, nombre) VALUES (6, 'Seis');
ROLLBACK TO sp1;
```

```
-- 3. ¿Qué devolverá esta consulta?
SELECT *
FROM producto;
-- -- "1 - Primero", "2 - Segundo", "3 - Tercero", "4 - Cuarto"
```

5. Transacciones

ACCESO CONCURRENTES A LOS DATOS

Cuando dos transacciones distintas intentan acceder concurrentemente a los mismos datos pueden ocurrir los siguientes problemas:

- **Dirty Read (Lectura sucia).** Sucede cuando una segunda transacción lee datos que están siendo modificados por una transacción antes de que haga COMMIT.
- **Nonrepeatable Read (Lectura No Repetible).** Se produce cuando una transacción vuelve a leer datos que leyó previamente y encuentra que han sido modificados por otra transacción.
- **Phantom Read (Lectura fantasma).** Este error ocurre cuando una transacción lee unos datos que no existían cuando se inició la transacción.

NIVELES DE AISLAMIENTO

Para evitar que sucedan los problemas de acceso concurrente del punto anterior podemos establecer diferentes niveles de aislamiento:

- **Read Uncommitted.** En este nivel no se realiza ningún bloqueo, por lo tanto, permite que sucedan los tres problemas.
- **Read Committed.** En este caso los datos leídos por una transacción pueden ser modificados por otras transacciones, por lo tanto, se pueden dar los problemas Phantom Read y Non Repeatable Read.
- **Repeatable Read.** En este nivel ningún registro leído con un SELECT puede ser modificado en otra transacción, por lo tanto, sólo puede suceder el problema del Phantom Read.
- **Serializable.** En este caso las transacciones se ejecutan unas detrás de otras, sin que exista la posibilidad de concurrencia.

El nivel de aislamiento que utiliza InnoDB **por defecto es Repeatable Read.**

5. Transacciones

NIVELES DE AISLAMIENTO

La sintaxis utilizada sería:

```
SET [GLOBAL | SESSION] TRANSACTION
    transaction_property [, transaction_property] ...

transaction_property:
    ISOLATION LEVEL level
    | READ WRITE
    | READ ONLY

level:
    REPEATABLE READ
    | READ COMMITTED
    | READ UNCOMMITTED
    | SERIALIZABLE
```

Para determinar el nivel de aislamiento existente (tanto a nivel global como a nivel de sesión), se puede comprobar a través de la variable del sistema ***tx_isolation***:

```
SELECT @@GLOBAL.tx_isolation, @@tx_isolation;
```

5. Transacciones

POLÍTICAS DE BLOQUEO

Cuando una transacción accede a los datos lo hace de forma exclusiva, de modo que **una transacción no podrá acceder a los datos que están siendo utilizados por una transacción hasta que ésta haya terminado**. El bloqueo de los datos se puede realizar a nivel de:

- Base de datos.
- Tabla.
- Fila.
- Columna.

InnoDB realiza **por defecto un bloqueo a nivel de fila**. En este caso, otras sesiones esperarán cuando realicen un intento de modificación o eliminación de las filas bloqueadas; en cambio, pueden modificar o eliminar otras filas no bloqueadas, o, de manera general, insertar nuevas. Además, esas otras sesiones pueden consultar la tabla con una sentencia *SELECT*, pero no ven las modificaciones no validadas de las otras sesiones.

Ejemplo: Simular que hay 2 usuarios que quieren acceder de forma concurrente a los mismos datos de una tabla. Para simular los dos usuarios hay que iniciar 2 terminales para conectarnos a un servidor de MariaDB.

Desde el **TERMINAL A** realizamos:

```
DROP DATABASE IF EXISTS test;
CREATE DATABASE test CHARACTER SET utf8mb4;
USE test;
CREATE TABLE cuentas (
  id INTEGER UNSIGNED PRIMARY KEY,
  saldo DECIMAL(11,2) CHECK (saldo >= 0)
);
INSERT INTO cuentas VALUES (1, 1000);
INSERT INTO cuentas VALUES (2, 2000);
INSERT INTO cuentas VALUES (3, 0);
SELECT * FROM cuentas;
-- "1 - 1000.00", "2 - 2000.00", "3 - 0.00"
-- 1. Ejecutamos una transacción para transferir dinero entre dos cuentas
```

```
START TRANSACTION;
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
UPDATE cuentas SET saldo = saldo + 100 WHERE id = 2;
SELECT * FROM cuentas;
```

-- En este momento "1 - 900.00", "2 - 2100.00", "3 - 0.00"
-- Esta transacción (ejecutándose en el terminal A todavía no ha finalizado porque no hemos ejecutado un COMMIT ni un ROLLBACK.

Ahora, desde el **TERMINAL B** ejecutamos las siguientes sentencias:

-- 1. Seleccionamos la base de datos

```
USE test;
```

-- 2. Observamos los datos que existen en la tabla cuentas

```
SELECT * FROM cuentas;
```

-- "1 - 1000.00", "2 - 2000.00", "3 - 0.00"

-- 3. Intentamos actualizar el saldo de una de las cuentas que está siendo utilizada en la transacción del terminal A

```
UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1;
```

-- En este momento, el terminal B se queda en estado Running y no deja ejecutar ninguna sentencia, ni siquiera de consulta de datos. Tras 30 segundos, aparece el mensaje de error:

3 12:02:16 UPDATE cuentas SET saldo = saldo - 100 WHERE id = 1 Error Code: 2013. Lost connection to MySQL server during query

-- Si tras este mensaje de error, se realiza:

```
SELECT * FROM cuentas;
```

-- "1 - 1000.00", "2 - 2000.00", "3 - 0.00"

Si ahora en el Terminal A ejecutamos un COMMIT para finalizar la transacción que estaba sin finalizar.

En A, consultando sus datos:

```
SELECT * FROM cuentas;
```

-- "1 - 900.00", "2 - 2100.00", "3 - 0.00"

En B, consultando sus datos:

```
SELECT * FROM cuentas;
```

-- "1 - 900.00", "2 - 2100.00", "3 - 0.00"

Si en B, en este momento, ejecutamos nuevamente la sentencia UPDATE anterior: UPDATE cuentas SET saldo = saldo - 100

WHERE id = 1; -- Ahora, sin problema, ejecuta dicha sentencia pues la transacción de A ya finalizó. SELECT * FROM cuentas; -- "1 - 800.00", "2 - 2100.00", "3 - 0.00"

Si queréis profundizar en el trabajo con transacciones se puede consultar:

- http://myy.haaga-helia.fi/~dbms/dbtechnet/download/SQL-Transactions_handbook_SP.pdf
- <https://mariadb.com/kb/en/transactions/>

***HACER EJERCICIOS DEL 23 AL 45**